M16 Multi-Agent : Topologie & Orchestration

Méthodologie Advanced

Architecturer des équipes d'agents pour les tâches complexes

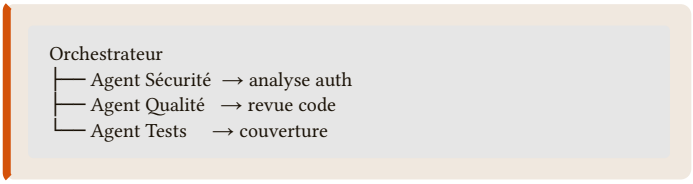
Quand passer en multi-agent

Passer au multi-agent quand le travail se décompose en périmètres indépendants, que les sorties peuvent être vérifiées séparément et que la coordination apporte plus que son coût. La taille du dépôt, le taux de contexte et le nombre d'agents ne donnent pas de seuil universel.

Signal	Agent unique	Équipe
Périmètre cohérent et séquentiel	Préférable	Surcoût inutile
Recherches indépendantes et vérifiables	Possible	Peut réduire le délai
Fichiers ou état partagés	Préférable	Risque de conflits
Reprise et synthèse complexes	Possible	Un orchestrateur explicite peut aider

Topologie étoile (Star)

L'orchestrateur central reçoit l'objectif global, le décompose en sous-tâches, délègue à des agents spécialisés, puis synthétise leurs résultats. C'est la topologie par défaut de Claude Code Agent Teams.



Dans cette organisation choisie, l'orchestrateur se concentre sur la planification et la délégation ; c'est une convention de rôle, pas une restriction d'outil. Son rôle est d'allouer le travail, de débloquer les dépendances, et de présenter une réponse unifiée.

Topologie pipeline

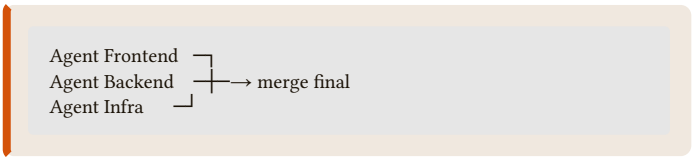
Les agents travaillent en séquence : la sortie de l'un devient l'entrée du suivant. Utile quand les étapes sont strictement ordonnées et que chaque résultat conditionne le suivant.



Avantage : chaque agent se concentre sur une phase précise, sans bruit des phases précédentes dans son contexte. Inconvénient : un blocage sur une étape arrête tout le pipeline.

Topologie parallèle

Des agents indépendants travaillent simultanément sur des modules sans dépendance entre eux. Cas typique : refactoring frontend, backend et infra en parallèle sur des fichiers non partagés.



Condition d'applicabilité : les modules doivent avoir zéro état partagé. Si les agents doivent constamment lire les sorties des autres ou modifier des fichiers communs, les conflits git et les messages de coordination annulent le gain.

Isolation et permissions par agent

Chaque agent peut recevoir un contexte et une whitelist d'outils distincts. Limiter un agent aux outils `Read`, `Glob`, `Grep` empêche toute modification accidentelle pendant la phase d'analyse.

Agent	Outils autorisés
Explorateur	Read, Glob, Grep
Planificateur	Read, Glob, Grep ; écritures du plan autorisées séparément
Implémenteur	Read, Edit, Bash, Write
Relecteur	Read, Glob, Grep

Activer les Agent Teams

```
# Variable d'environnement (session courante)
export CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1
claude

# Configuration persistante ~/.claude/settings.json
{
  "env": {
    "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1"
  }
}
```

Prérequis : Claude Code v2.1.32+, modèle accessible au compte (différent selon le coéquipier si besoin). Git est nécessaire à l'isolation par worktree, pas à la messagerie des tâches. `Shift+Down` passe entre agents dans le même processus.

Contrôles de boucle

Définir avant le lancement un budget, une condition d'arrêt, un état de reprise et le responsable de la synthèse. Ajouter un relecteur indépendant lorsque le changement est conséquent ou que le même agent ne peut pas produire et vérifier un résultat de manière crédible.

Limiter le travail lancé à la capacité de vérification ; commencer avec la plus petite équipe couvrant des périmètres indépendants. Ajouter un participant seulement si les mesures de délai, de conflits, de corrections humaines et de coût par tâche acceptée justifient la coordination supplémentaire. Le protocole d'essai permet de documenter cette décision.